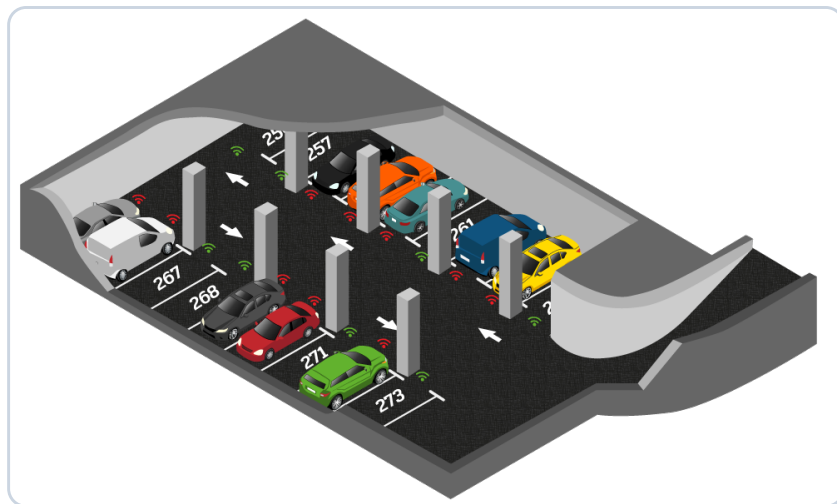


Smart Parking

Konzeption und Umsetzung des Verwaltungs-Dashboards für ein intelligentes Parkhaus



Quelle: https://iot-shop.de/aot_website_iot_shop/static/src/img/smart-parking/parkhaus.png

Studiengang:	Master Informatik
Verfasser:	Philipp Zausinger
Dozent/in:	Prof. Dr. Jube
Abgabe:	19. Juli 2026

Inhaltsverzeichnis

1	Projektbeschreibung	3
1.1	Zentrale Use Cases des Dashboards	3
1.2	Architekturüberblick	3
2	Frontend-Architektur	4
2.1	Technologie-Stack	4
2.2	Projektstruktur	4
2.3	API-Anbindung und Datenfluss	4
3	Die drei Dashboards	4
3.1	Übersichts-Dashboard	5
3.2	Bezahl-Dashboard	5
3.3	Admin-Dashboard	6
4	Tests	6
5	Fazit, offene Aspekte und Ausblick	7
5.1	Fazit	7
5.2	Offene Aspekte	7
5.3	Ausblick	7

1 Projektbeschreibung

Im Rahmen des Projekts „Smart Parking“ wurde im Team ein Modell eines intelligenten Parkhauses mit Kennzeichenerkennung, ereignisbasierter Steuerung und mehreren Web-Dashboards umgesetzt. Einfahrt, Stellplatzbelegung, Zahlung und Ausfahrt werden über MQTT an ein Node-RED-Backend gemeldet, dort verarbeitet und in einer SQLite-Datenbank gespeichert. Backend, Datenbank und Hardwarekomponenten wurden von meinen Projektpartnern umgesetzt und sind entsprechend in deren Dokumentationen beschrieben. Der vorliegende Bericht behandelt den von mir umgesetzten Teil: das **Verwaltungs-Dashboard**, ein React-Frontend mit drei eigenständigen Ansichten für Übersicht, Bezahlung und Administration, das über REST-Schnittstellen mit dem Node-RED-Backend kommuniziert.

1.1 Zentrale Use Cases des Dashboards

- Belegung aller Stellplätze je Etage in Echtzeit einsehen,
- aktive Parkvorgänge und zuletzt protokollierte Ereignisse überblicken,
- ein Kennzeichen suchen, den fälligen Betrag anzeigen und bezahlen,
- Kamera-Livebilder, RFID-Berechtigungen und offene Sitzungen administrieren,
- Ereignisprotokoll nach Typ, Kennzeichen und Zeitraum filtern.

1.2 Architekturüberblick

Abbildung 1 zeigt, wie sich das Dashboard in das Gesamtsystem einordnet. Die drei Ansichten laufen als einzelne *React-Komponenten* innerhalb einer Single-Page-Application und greifen über eine gemeinsame API-Schicht auf das Node-RED-Backend zu. MQTT-Kommunikation und Hardwareanbindung liegen außerhalb des Dashboards und werden von den Backend-Flows meines Projektteams verarbeitet.

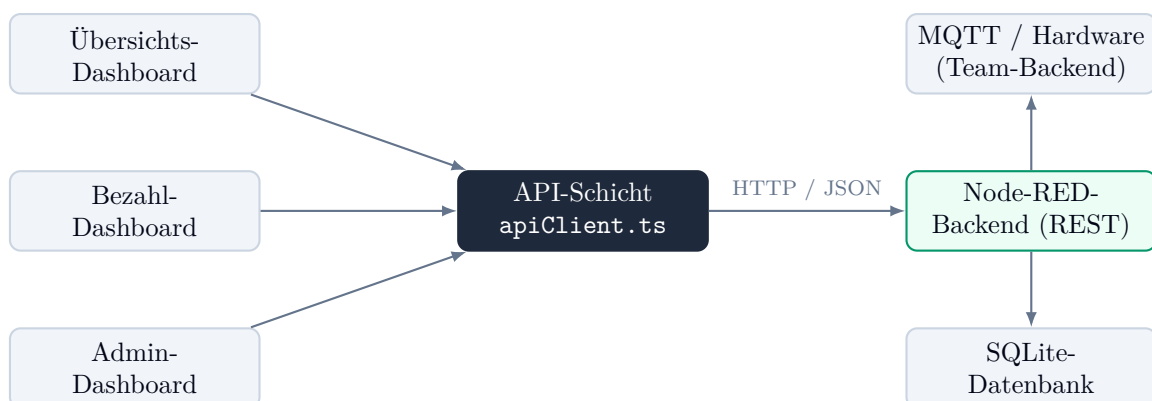


Abbildung 1: Einordnung des Dashboards in die Gesamtarchitektur des Smart-Parking-Systems

2 Frontend-Architektur

2.1 Technologie-Stack

Das Dashboard ist als React-Single-Page-Application in TypeScript umgesetzt und wird mit Vite gebaut. Für das Styling wird TailwindCSS eingesetzt, wodurch sich ein konsistentes Erscheinungsbild über alle drei Ansichten hinweg umsetzen ließ, ohne eine eigene CSS-Architektur pflegen zu müssen.

Baustein	Verwendung
React 18 + TypeScript	Komponentenbasierte Oberfläche, typsichere Datenmodelle
Vite	Entwicklungsserver und Produktions-Build
TailwindCSS 4	Utility-first Styling, einheitliches Design
Fetch API + eigener Hook	Datenabruf, Live-Aktualisierung, Fehlerbehandlung

Tabelle 1: Eingesetzter Technologie-Stack des Dashboards

2.2 Projektstruktur

Der Quellcode ist nach fachlicher Zuständigkeit gegliedert: `pages/` enthält die drei Dashboards und die Startseite, `api/` die REST-Anbindung, `hooks/` den Datenabruf-Hook, `components/ui` wiederverwendbare Bausteine und `types/` die gemeinsamen TypeScript-Typen. Zusätzlich existiert ein Ordner `backend/`, der während der Entwicklung als lokaler Mock-Datenspeicher diente, solange kein Node-RED-Server erreichbar war (siehe Abschnitt 4).

2.3 API-Anbindung und Datenfluss

Alle Netzwerkzugriffe laufen über einen zentralen Wrapper `apiClient.ts`. Er ergänzt Basis-URL und Header, wandelt Antworten in JSON um und wirft im Fehlerfall eine einheitliche `ApiError` mit HTTP-Status. Ein Status von 0 kennzeichnet dabei speziell einen nicht erreichbaren Server, wodurch alle Dashboards Netzwerk- und Anwendungsfehler unterschiedlich behandeln können. Darauf aufbauend bündeln `mapApi.ts`, `paymentsApi.ts` und `adminApi.ts` jeweils die Endpunkte einer fachlichen Domäne.

Für den Datenabruf innerhalb der Komponenten wurde der Hook `useFetch` entwickelt. Er kapselt Ladezustand, Fehlerzustand und ein optionales Intervall (`pollingMs`), über das Daten automatisch neu geladen werden. Das Übersichts-Dashboard nutzt dies beispielsweise, um Stellplätze, Kennzahlen, offene Sitzungen und Ereignisprotokoll sekundlich zu aktualisieren, ohne dass jede Komponente eigene Lade- und Fehlerlogik implementieren muss.

3 Die drei Dashboards

Von der Startseite aus gelangt man über drei Kacheln zu den jeweiligen Dashboards (vgl. `HomePage.tsx`). Jedes Dashboard ist eigenständig aufgebaut und für ein anderes Einsatzszenario gedacht.

3.1 Übersichts-Dashboard

Das Übersichts-Dashboard richtet sich an Betreiber und zeigt den aktuellen Systemzustand auf einen Blick: Gesamtzahl, belegte und freie Stellplätze sowie die Auslastung in Prozent, farblich gestaffelt von grün über gelb bis rot. Darunter visualisiert eine Etagenansicht jeden einzelnen Stellplatz als farbiges Feld (belegt/frei). Es folgen eine Tabelle der aktiven Parkvorgänge mit Kennzeichen, Einfahrtszeit, bisheriger Parkdauer und fälligem Betrag sowie eine laufend aktualisierte Liste der letzten protokollierten Ereignisse. Während des Ladens werden Platzhalter-Skelette angezeigt, bei Verbindungsfehlern erscheint ein Hinweis mit Wiederholen-Schaltfläche.

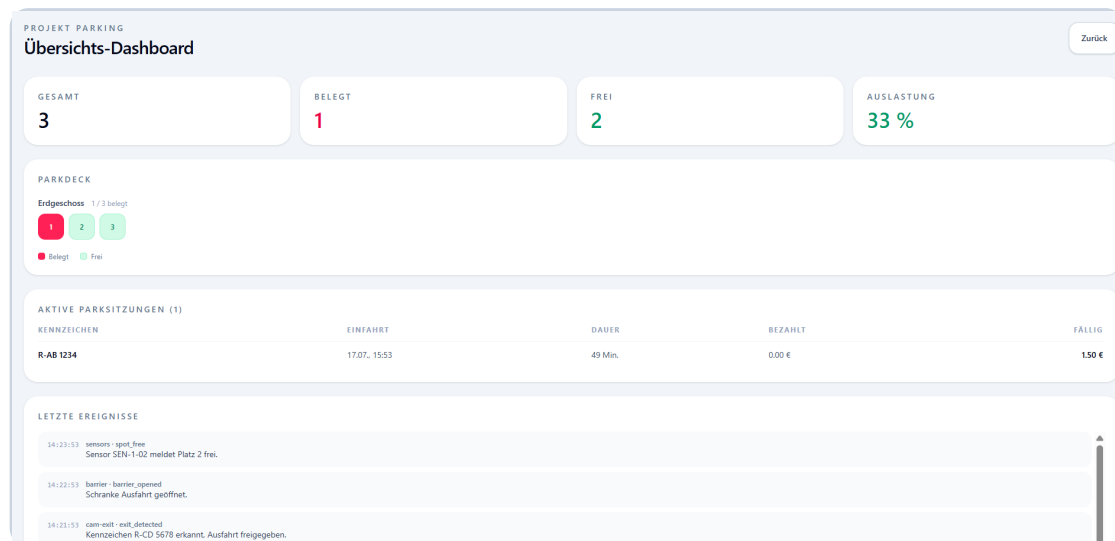


Abbildung 2: Kennzahlen, Etagenbelegung, aktive Sitzungen und Ereignisprotokoll im Übersichts-Dashboard

3.2 Bezahl-Dashboard

Das Bezahl-Dashboard ist für einen Terminal-/Kiosk-Betrieb ausgelegt und kommt ohne physische Tastatur aus: Ein Bildschirm-Tastenfeld mit Buchstaben und Ziffern erlaubt die Eingabe des Kennzeichens, ergänzt um eine laufende Uhrzeitanzeige. Nach Eingabe wird die zugehörige Parksitzung über `fetchSessionByPlate` gesucht; das Interface unterscheidet dabei explizit zwischen „Kennzeichen nicht gefunden“ (HTTP 404), „Server nicht erreichbar“ sowie sonstigen Fehlern und gibt entsprechendes Feedback.

Wird eine Sitzung gefunden, wechselt die Anwendung zur Zahlungsansicht (`payment_price.tsx`): Sie zeigt Kennzeichen, Einfahrtszeit sowie den offenen Betrag und öffnet über die Schaltfläche „Bezahlen“ ein Modal zur Auswahl der Zahlungsart (Karte oder PayPal, simuliert). Nach Bestätigung wird die Zahlung über `patchPaymentSession` gebucht; bei Erfolg erscheint ein Bestätigungsbildschirm mit Hinweis auf ein 15-minütiges Ausfahrtsfenster, der nach einer Minute automatisch zur Kennzeichen-Eingabe zurückkehrt.

Freitag, 17.07.2026, 14:44:46 Zurück

BEZAHL-DASHBOARD

Kennzeichen eingeben
Gib das Kennzeichen ein oder nutze die Tasten für Buchstaben und Zahlen.

KENNZEICHEN

R-AB 1234

A B C D E F G H I
 J K L M N O P Q R
 S T U V W X Y Z -
 0 1 2 3 4
 5 6 7 8 9

Leeren Bezahlen

Abbildung 3: Kennzeicheneingabe über das Bildschirm-Tastenfeld im Bezahl-Dashboard

3.3 Admin-Dashboard

Das Admin-Dashboard bündelt technische und organisatorische Funktionen für Betreiberpersonal. Es zeigt die Live-Bilder der Einfahrts- und Ausfahrtskamera als MJPEG-Stream inklusive Fehlererkennung und manuellem Neuverbinden, verwaltet RFID-Administratoren (Anlegen, Aktivieren/ Deaktivieren), erlaubt das manuelle Bearbeiten offener Parksitzungen sowie das Filtern und Durchsuchen des vollständigen Ereignisprotokolls nach Ereignistyp, Kennzeichen und Zeitraum. Die hinterlegten Tarifregeln werden aktuell noch lokal im Frontend vorgehalten, da hierfür bislang keine eigene REST-Schnittstelle existiert.

PROJECT PARKING **Admin-Dashboard** Zurück

LIVE-KAMERAS

EINFAHRT **AUSFAHRT**

OFFENE PARKSESSIONS (1)

ID	KENNZEICHEN	EINFAHRT	FÄLLIG	BEZAHLT	
101	R-AB 1234	17.07.2026, 15:03	1.50 €	0.00 €	Beenden

RFID-ADMINIS (3)

NAME	RFID-ID	ROLLE	STATUS
------	---------	-------	--------

Neue Session

Abbildung 4: Kamera-Livebilder, RFID-Verwaltung und Ereignisprotokoll im Admin-Dashboard

4 Tests

Da während eines Großteils der Entwicklungszeit kein durchgehend laufendes Node-RED-Backend zur Verfügung stand, wurde das Dashboard zunächst gegen lokale

Mock-Datenquellen im Ordner `backend/` entwickelt und getestet, die realistische Beispieldaten für Parkvorgänge, Stellplätze und Ereignisse bereitstellen. Dadurch konnten alle drei Ansichten unabhängig vom Backend-Fortschritt der Projektpartner entwickelt und mit realistischen Zuständen (freie/belegte Plätze, offene/bezahlte Sitzungen) durchgespielt werden. Anschließend wurde gegen das echte Node-RED-Backend getestet. Mit dem Python-Skript `mqtt_sim.py` ließen sich Einfahrts- und Ausfahrtsereignisse simulieren und unmittelbar im Übersichts-Dashboard nachverfolgen. Zusätzlich wurden Fehlerpfade gezielt geprüft, etwa das Verhalten der Oberfläche bei nicht erreichbarem Server oder bei einem nicht gefundenen Kennzeichen im Bezahl-Dashboard.

5 Fazit, offene Aspekte und Ausblick

5.1 Fazit

Es ist ein funktionsfähiges Verwaltungs-Dashboard entstanden, das drei klar getrennte Nutzungsszenarien – Überblick, Bezahlung am Terminal und Administration – in einer gemeinsamen Anwendung mit einheitlichem Design abbildet. Durch den zentralen `useFetch`-Hook und die konsistente Fehlerbehandlung über `ApiError` verhält sich die Oberfläche in allen drei Ansichten vorhersehbar, auch wenn das Backend zeitweise nicht erreichbar ist. Die frühzeitige Entwicklung gegen Mock-Daten hat sich als sinnvoll erwiesen, da Frontend und Backend so weitgehend unabhängig voneinander fortschreiten konnten.

5.2 Offene Aspekte

Das Admin-Dashboard verfügt aktuell über keinerlei Zugriffsschutz; jede Person mit Netzwerkzugriff kann Sitzungen bearbeiten oder RFID-Berechtigungen ändern. Die Aktualisierung per Polling im Sekundentakt ist einfach umzusetzen, verursacht bei mehreren gleichzeitig geöffneten Dashboards jedoch unnötige Serverlast. Außerdem sind die Tarifregeln bislang nicht über die REST-Schnittstelle abrufbar und daher im Frontend hinterlegt statt zentral im Backend gepflegt zu werden.

5.3 Ausblick

Für einen produktiven Einsatz wären eine Benutzeranmeldung mit Rollenprüfung für das Admin-Dashboard sowie eine Ablösung des Pollings durch serverseitige Push-Updates (z. B. über WebSockets) sinnvolle nächste Schritte. Ebenso sollten die Tarifregeln über eine eigene REST-Schnittstelle verwaltet werden, damit Änderungen nicht mehr im Frontend-Code vorgenommen werden müssen.

Literatur

- [1] React Documentation: *Describing the UI*. <https://react.dev/learn>, abgerufen am 19. Juli 2026.
- [2] Vite Documentation: *Getting Started*. <https://vitejs.dev/guide/>, abgerufen am 19. Juli 2026.
- [3] Tailwind Labs: *Tailwind CSS Documentation*. <https://tailwindcss.com/docs>, abgerufen am 19. Juli 2026.

KI-Klausel

ChatGPT bzw. Claude wurden punktuell zur sprachlichen Überarbeitung einzelner Textstellen sowie zur Erstellung des Architekturdiagramms eingesetzt. Konzeption und Umsetzung des Dashboards sowie die fachlichen Inhalte dieser Dokumentation stammen vom Verfasser. Sämtliche Vorschläge wurden eigenständig geprüft und bei Bedarf angepasst.